

CatCodeDidi Project Setup Guide

Step-by-step roadmap to build the CatCodeDidi project from scratch.

Stage 1: Install the Tools

- Install Python 3.10+.
- Install PortAudio for microphone support on macOS/Linux.
- Verify the Python version before continuing.

Stage 2: Create the Project Folder

- Create a new folder named CatCodeDidi.
- Set up the project structure with the main Python modules and tests.
- Keep the project clean and organized from the start.

Stage 3: Create the Virtual Environment

- Run `python3 -m venv .venv`.
- Activate the environment using the OS-specific command.
- Keep dependencies isolated in the virtual environment.

Stage 4: Install Dependencies

- Install SpeechRecognition, PyAudio, gTTS, playsound3, and Gemini SDK packages.
- Install test dependencies separately if needed.
- Use requirements.txt to keep setup repeatable.

Stage 5: Configure the Assistant

- Create config.py for the bot name and language.
- Set up the assistant personality and default settings.
- Store app-specific constants in one place.

Stage 6: Build the Speech Layer

- Listen to the microphone and convert audio to text.
- Use text-to-speech to make the assistant speak back.
- Handle mute and unmute behavior cleanly.

Stage 7: Add Command Logic

- Open and close apps.
- Capture screenshots.
- Detect creator questions and user-defined commands.
- Handle OS differences for macOS, Linux, and Windows.

Stage 8: Connect Gemini AI

- Create a Gemini client using the API key.
- Load the key from .env.
- Send general questions to Gemini and return the response.

Stage 9: Build the Main App Loop

- Greet the user.
- Listen for input in a loop.
- Check exit commands and route user requests correctly.
- Print and speak the final response.

Stage 10: Add Wake Word and Personality

- Add a wake phrase like Didi or Cat Code.
- Customize greeting messages by time of day.
- Make the assistant feel more natural and engaging.

Stage 11: Write Tests

- Write tests for commands, speech, AI responses, and personality.
- Run pytest to validate the project behavior.
- Fix errors before expanding the app.

Stage 12: Run, Debug, and Improve

- Run `python main.py`.
- Test real commands and fix issues.
- Improve features gradually after the project is working.

Final outcome: a working Python voice assistant with speech input, speech output, app commands, AI responses, and testing.